

Ticket To Ride London Design Document

Author:

Alan Sebastian

The purpose of this document is to provide you with a guideline for writing the software design document for your project.

Points to remember:

- *Please include "Sprint 3" and your full name in the file name of the PDF document.*
- *Content is important, not the volume. **Another team should be able to develop this system from only this document.***
- *Pay attention to details.*
- *Completeness and consistency will be rewarded.*
- *Readability is important.*
- *Remove the explanations shown in blue before submitting.*

Document Revision History

Date	Version	Description	Author
24/05/2026	0.0	Initial draft	
30/05/2026	0.1	Added design rationale	Alan Sebastian
30/05/2026	0.2	Added class diagrams	Alan Sebastian
31/05/2026	0.3	Added Antipatterns and refactoring information	Alan Sebastian

1 Introduction

This document presents the reasoning behind the Sprint 3 digital implementation of Ticket to Ride London, outlining the updated system architecture, gameplay extensions and key design decisions used throughout the development.

The core problem addressed in this project is the translation of a complex, rule based physical board game into a consistent, and well-defined digital system. Ticket to Ride London contains multiple interacting gameplay components including players, routes, transport cards, destination tickets, scoring systems and turn management rules. Sprint 3 further extends this complexity through the introduction of ferry routes, save/load functionality and custom gameplay mechanics. Modelling these elements in software requires careful consideration to ensure that game rules are enforced correctly, player actions remain constrained appropriately, and the digital gameplay experience remains faithful to the original board game.

The intended audience for this document includes developers, tutors, assessors, and other stakeholders involved in the project. The document serves both as a technical reference for implementation and as a justification of the architectural and design decisions made during development.

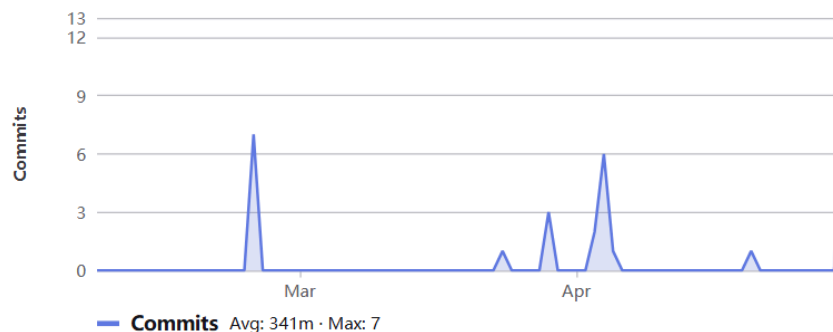
The document is organised into several sections. First, the updated class diagram and Sprint 3 extensions are presented. This is followed by a discussion of identified antipatterns and the refactoring techniques applied. Finally, the design rationales explain important class, relationship, quality attribute, and design pattern decisions made during development.

GitLab Repository:

<https://git.infotech.monash.edu/FIT3077/fit3077-s1-2026/assignment-individual/aseb0007>

Alan Sebastian

28 commits (aseb0007@student.monash.edu)



2 Updated Class Diagram

The three extensions implemented for Sprint 3 are:

1. Required Extension - Ferry Routes: Ferry routes introduce special route claiming rules involving locomotive requirements. To claim a ferry route, players must use a specified number of locomotive cards in addition to the normal coloured cards required for the route. Players may also use extra locomotives as wildcards and, if insufficient locomotives are available, any three standard cards may be substituted for one missing locomotive. This extension required changes to the route claiming logic and the visual representation of routes
2. Pick From The List Extension - Save/Load System: The game now supports saving a running game to an external text file and restoring a previously saved game state. The system stores gameplay information such as player scores, transport cards, destination tickets, claimed routes, buses, and turn information. A dedicated GameSaveManager class was introduced to handle persistence responsibilities separately from gameplay control.
3. Self Defined Extension - Landmark Routes: LandMark routes are special routes that trigger additional gameplay effects when claimed. Unlike standard routes, landmark routes provide unique interactions such as selectable rewards, bonus points or player interaction mechanics. Implemented landmark effects include STEAL_CARD, POINTS_OR_CARDS and DETECTIVE_CHOICE.

Ticket To Ride London Class Diagram
Alan Sebastian, Neil Pereira, Scott Nguyen

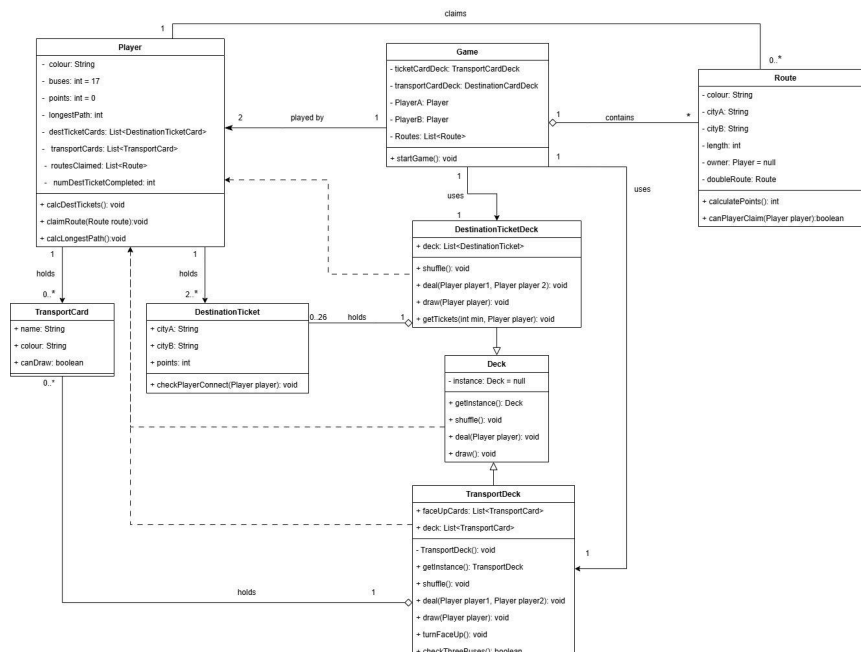


Figure 1.1 - Initial class diagram focussed on game logic.

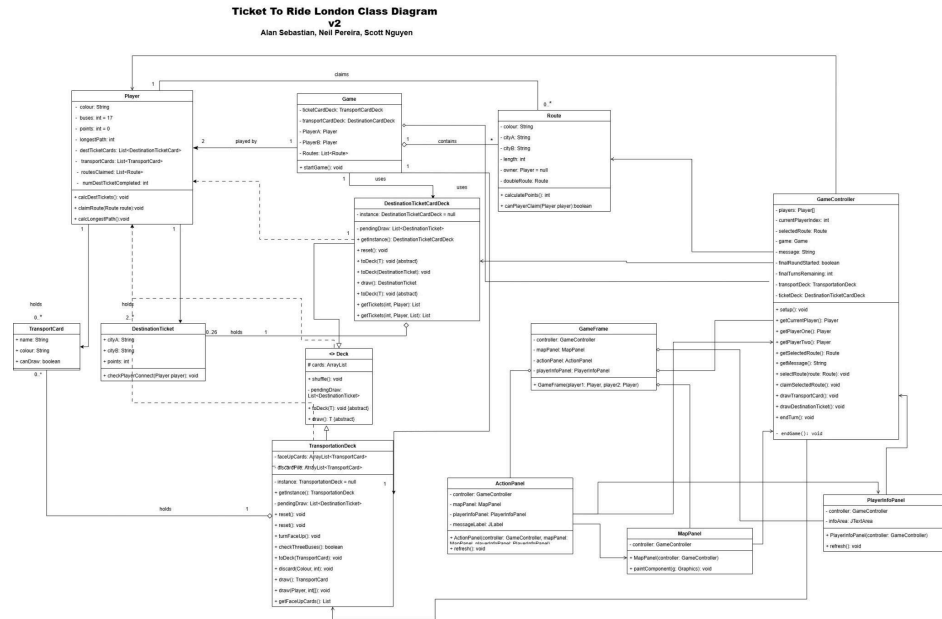
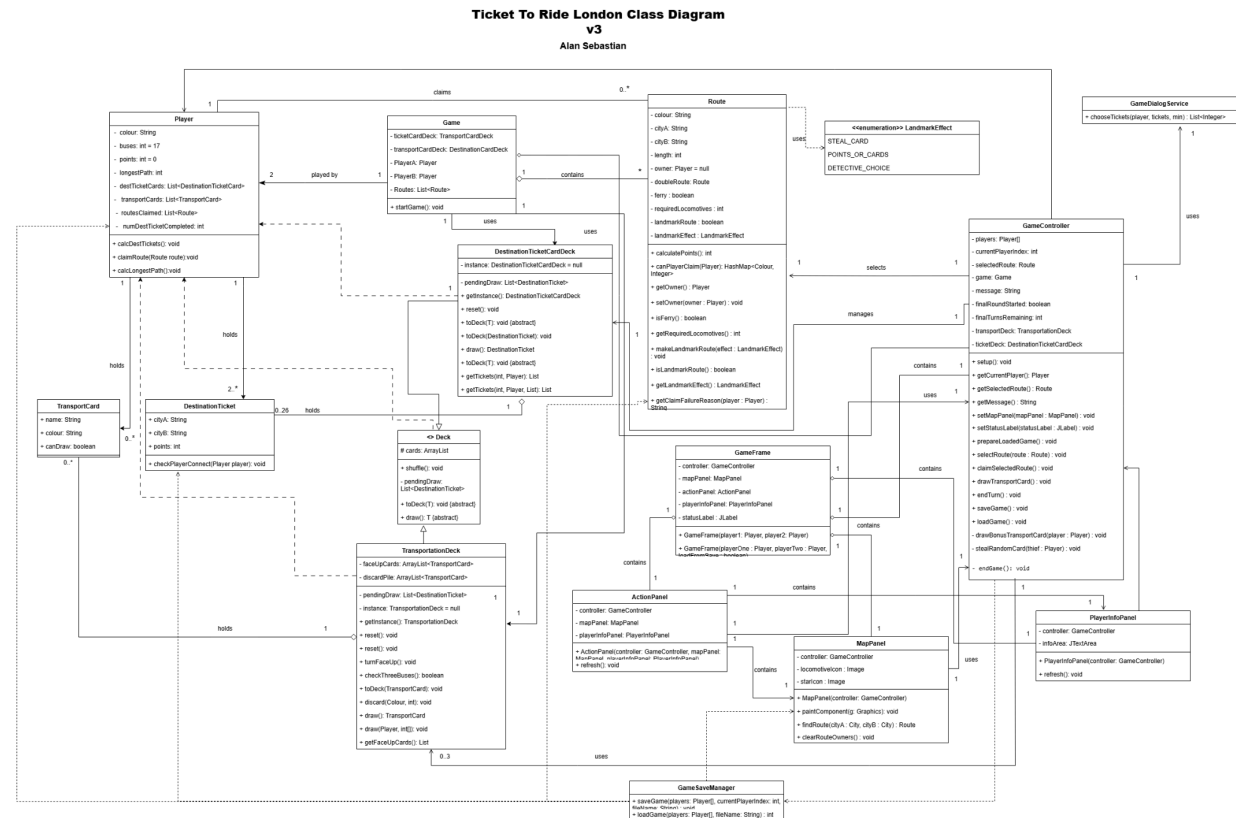


Figure 1.2 - Second iteration of the class diagram for sprint 2 requirements.

Figure 1.3 - Final Class Diagram with Sprint 3 requirements and additions considered
Compared to Sprint 2, the main changes include:

- Extension of the Route class to support ferry and landmark behaviour through additional attributes, validation logic, and route effect handling.
- Introduction of the GameSaveManager class to manage game saving and loading responsibilities.
- Addition of the LandmarkEffect enumeration to represent valid landmark route behaviours in a controlled and type-safe manner.
- Updates to GameController, GameFrame, and MapPanel to support save/load functionality, gameplay messaging, landmark activation, and special route visualisation.
- Updated relationships, multiplicities, dependencies, and cardinalities reflecting the Sprint 3 implementation and revised architecture.
- Introduction of the GameDialogService class to separate destination ticket dialog handling from GameController, improving separation of concerns and reducing controller responsibilities.

3 Antipatterns and Refactoring

1. Dead Code/ Duplicated Responsibility

In Sprint 2, GameController contained unused helper methods `dealStartingCards()` and `giveRandomTransportCard()`, which duplicated the responsibility of the transportation deck. This created dead code and made the setup process less clear because there appeared to be two possible ways to deal starting cards. I applied the Remove Dead Code refactoring technique by deleting these methods and relying only on `transportDeck.deal(player)` inside `setup()`. This improved cohesion because card dealing remains the responsibility of the deck, while GameController only coordinates setup.

2. God Controller - GameController

GameController handled both game coordination logic and user interface dialog logic. In particular, the `chooseTickets()` method created Swing checkboxes, displayed `JOptionPane` dialogs, and validated ticket selections, even though these behaviours are presentation responsibilities. This reduced cohesion and tightly coupled the controller to the UI layer. I applied the Extract Class refactoring technique by creating a new `GameDialogService` class and moving the ticket selection dialog logic into it. GameController now delegates this behaviour through `dialogService.chooseTickets(...)`. This improves separation of concerns, readability, and maintainability by reducing the controller's responsibilities.

3. Primitive Obsession - Landmark Route Effects

Another antipattern identified was primitive obsession in the landmark route extension. Landmark effects were originally represented using raw string values such as `"STEAL_CARD"`, `"POINTS_OR_CARDS"`, and `"DETECTIVE_CHOICE"`. This was problematic because string-based logic is fragile — a typo could still compile but cause incorrect behaviour at runtime. I addressed this using the Replace Magic String with Enum refactoring technique by introducing a `LandmarkEffect` enum. Landmark routes now store a defined effect type instead of unrestricted strings. This improves type safety,

readability, and maintainability, while also making new landmark effects easier to add in a controlled way.

4 Design Rationales

4.1 Classes, Responsibilities and Responsibility Reassignment

There were several new classes and structures introduced during Sprint 3 to support the required extensions while attempting to keep responsibilities separated.

One of the main additions was the `GameSaveManager` class. This class was introduced to handle the saving and loading gameplay state, including players, transport cards, destination tickets, claimed routes, scores, buses, and turn information. An alternative approach would have been to place this behaviour directly inside `GameController`, since the controller already has access to gameplay information. However, by doing this it would have given the controller responsibility for both gameplay coordination and file management. These concerns serve different purposes, so combining them would lower cohesion and unnecessarily increase the controller's complexity. Introducing a `GameSaveManager` makes persistence logic and gameplay control separate, resulting in cleaner code that is easier to then maintain and extend in future.

The `Route` class also required significant changes during Sprint 3 through new attributes being introduced. These included `ferry`, `requiredLocomotive`, `landmarkRoute`, and `landmarkEffect` to support the ferry and landmark extensions. These responsibilities could potentially have been placed into specialised subclasses such as `FerryRoute` or `LandmarkRoute`. However, ferries and landmarks still behave fundamentally as routes: they connect cities, have lengths, can be claimed by players and award points. This meant that extending the existing `Route` class reduced duplicated behaviour and kept route related logic within a single gameplay model.

The `LandmarkEffect` enumeration was also added to represent the available landmark behaviours. This replaced raw string values previously used to represent effects. While the route class could technically have continued storing strings, an enumeration was chosen because landmark effects represent a controlled set of valid values rather than just having any random string be able to be a value. Using an enum improves readability, type safety, and reduces the risk of invalid or misspelled effect values.

Sprint 3 also required some responsibility reassignment from the Sprint 2 implementation as antipatterns were discovered. In sprint 2, `GameController` handled dialog logic related to ticket selection but this functionality mainly involved UI elements such as Swing components, validation and dialog presentation rather than the core gameplay logic. Moving it into `GameDialogService` kept the controller focused on coordinating gameplay while giving any UI related behaviour a clearer and more appropriate place. However,

not all UI related behaviour was moved out of the controller since some parts were still closely tied to gameplay coordination and interface updates. This was a practical design trade-off that improved cohesion and separation of concerns while avoiding unnecessary changes to the overall structure.

4.2 Alternative Designs Considered

There were many alternative designs that were considered during the implementation.

For the ferry extension, one alternative design was creating a dedicated FerryRoute subclass through inheritance. This design would separate ferry behaviour from standard route behaviour. However, ferry routes still share most functionality with standard routes, including ownership, scoring, city handling, and basic claiming behaviour. Introducing a subclass would therefore duplicate a large amount of logic or require complex inheritance relationships for relatively small behavioural differences. Extending the existing `Route` class was ultimately considered simpler and easier to maintain.

A similar decision was made for landmark routes. Another possible design involved creating a separate hierarchy of landmark route subclasses, each representing a different landmark effect. This approach would provide stronger separation between effect behaviours but would significantly increase the number of classes in the design. Because the implemented landmark effects only alter a limited portion of route behaviour, subclassing was considered unnecessarily complex for the scale of the project.

For save/load functionality, the most obvious alternative design was implementing persistence directly inside `GameController`. This would avoid introducing an additional class. However, it would tightly couple gameplay logic with file management and make future modifications to the save format more difficult. The dedicated GameSaveManager approach provided cleaner separation of concerns and better maintainability.

An additional alternative considered for the landmark system was using the Strategy design pattern. Different landmark effects could be implemented as separate strategy objects responsible for their own behaviour. This design would reduce conditional logic and support highly extensible landmark functionality. However, for the limited number of effects implemented in Sprint 3, this solution was considered unnecessary and would introduce additional abstraction and boilerplate code beyond what was required.

4.3 Quality Attributes and Trade-offs

One of the main quality attributes considered throughout Sprint 3 was maintainability. This was supported through tactics such as separation of concerns and responsibility decomposition. The introduction of GameSaveManager improves this by keeping persistence logic separate from gameplay coordination rather than embedding file

handling inside GameController. A scenario demonstrating this is if additional gameplay data needs to be saved in a future sprint, for example tracking statistics or extended player state, modifications can be made largely within the manager without requiring substantial changes to the controller or UI classes. The LandmarkEffect enumeration also contributes to maintainability since valid effects are defined in a single controlled location rather than scattered as string literals across the codebase, which reduces the chance of inconsistent values and makes adding new landmark behaviours more straightforward.

Extensibility was also considered as a quality attribute throughout the sprint. This was supported through controlled extension points and encapsulation of specialised behaviour. The system was designed so that additional landmark effects, route behaviours, or saved information could be introduced without major architectural restructuring. A concrete scenario here is that a new landmark effect can be added by defining it in the enumeration and writing the corresponding controller logic without redesigning how routes work overall, which demonstrates that the extension point was built into the design rather than being an afterthought.

Several tradeoffs were accepted as part of these decisions. Extending Route rather than introducing subclasses improves reuse and keeps claiming logic consolidated, but the class is noticeably larger and more complex as a result. Similarly introducing GameSaveManager and GameDialogService as dedicated helper classes improves separation of concerns but increases the total number of classes within the overall design, which adds some navigational overhead when reading through the codebase.

Performance was not treated as a primary concern for this project. The game operates with relatively small datasets and runs as a local standalone application, so there was no realistic scenario where performance would become a bottleneck under normal use. Design decisions were therefore weighted towards readability, maintainability, and architectural clarity rather than optimisation, which was considered an appropriate tradeoff given the scope and execution environment of the project.

4.4 Design Patterns and Pattern Considerations

There were no new design patterns introduced as a part of the Sprint 3 extensions. The singleton pattern that was inherited from Sprint 2 remains in use for the TransportationDeck and DestinationTicketCardDeck classes, which I considered appropriate as the physical board game contains exactly one shared transportation deck and one shared destination ticket deck at all times. Using a single shared instance for these keeps access to deck state straightforward and reflects how the game actually works. The known drawbacks of singletons such as tighter coupling, hidden dependencies and reduced testability were considered and to keep these manageable the pattern was not used anywhere else besides deck management.

Since no new patterns were introduced for the Sprint 3 extensions, three candidates that were considered but rejected are discussed below.

The strategy pattern was considered during the design of the landmark route extension. Each landmark effect could have been modelled to be a separate behaviour object by implementing a shared interface, which would make adding new effects later a matter of introducing a new strategy class rather than modifying existing logic. For a system with a large number of complex effects this may have been a good approach, but with only three implemented landmark behaviours so far, the overhead of defining interfaces and separate classes for each one was thought to be unnecessary for what the extension actually needed and so an enum based approach was used instead.

The Observer Pattern was also considered when thinking about how gameplay events and UI updates were being handled. Rather than having the controller manually trigger UI refreshes after each state change, UI components could instead observe the game state and update themselves whenever something relevant changes, which would reduce how tightly the frontend and backend are coupled to each other. The pattern would have been cleaner but would also have introduced a level of complexity, with listeners, event types and notification logic which was not justified given the relatively modest scope of the messaging and display requirements in Sprint 3.

The Factory Method Pattern was considered for route creation, since Sprint 3 introduced two specialised route types that could plausibly have warranted their own creation logic. A factory approach would centralise how different route objects are constructed and make it easier to extend the system if more route variants were added in future. In the current implementation though route creation is simple and the number of route types is small enough that adding a factory layer would increase the complexity of the codebase without providing any real practical benefit for the scale of the project.

5 Supplementary Documentation

Several minor implementation details and development considerations are documented here for completeness.

During Sprint 3, iterative development resulted in small differences between the initial UML design and the final source code implementation. These differences were not substantial and mainly involved minor method placement, helper method extraction, and refinements made during implementation and testing. The overall architecture, class responsibilities, and extension design remained consistent with the presented class diagram.

Custom graphical assets were used for several Sprint 3 gameplay indicators, including locomotive and landmark route icons, to improve the clarity and visual representation of special route behaviour within the map interface.

Artificial intelligence tools were used during development as a support tool for brainstorming, debugging, reviewing design decisions, improving documentation quality, and discussing alternative implementation approaches. All architectural decisions, code modifications, refactoring choices, extension implementations, and final written content were reviewed, adapted, and integrated by myself to fit the requirements and design of the project.

No additional supplementary information is required beyond the material presented in this document.